CHAPTER 7: MODEL, CONTEXT WINDOW, CONTEXT STRATEGY

Summary: *This is the most heavily documented material on the exam, which means the questions can be precise and the wrong answers can be specific. Three things decide most of it: what actually consumes the window, why filling it makes quality worse rather than merely expensive, and when to load material up front against fetching it on demand. Model choice sits underneath all of that, and the defensible way to make it is to measure rather than to argue.*

Everything in the Request Counts, Including the Part You Cached

Start with the inventory, because a surprising number of design errors are just bad accounting.

Everything in a request counts toward the context window: the system prompt, every message in the conversation along with its tool results, images and documents, the tool definitions, and the output generated for the turn. Each response reports what it consumed in its usage field.[1]

Two entries there catch people out.

Tool definitions occupy the window. A catalogue of 60 tools is paid for on every request whether or not any of them is called. This is the fact that makes the progressive-discovery question later in this chapter a real trade-off rather than a preference.

Cached prompt prefixes still occupy the window. Caching changes what you are charged for those tokens. It does not change whether they count.[1] A design that caches a large preamble and concludes it has therefore freed up room has made an arithmetic error, and it will find out at the cliff rather than at the invoice.

The Working-Memory Cliff

The context window is working memory rather than storage, and Anthropic is direct that more context is not automatically better, because **accuracy and recall degrade as the token count grows**.[1]

Name it: the **working-memory cliff** is the point where adding material stops helping and starts costing you recall of the material already there. It has two properties that make it dangerous.

It is not an error. Nothing fails, nothing throws, no status code changes. The system returns a fluent answer that has quietly stopped attending to something in the middle of what you gave it.

It is not uniform. Recall does not degrade evenly across the window, which is why the characteristic symptom is that material near the middle of a long collection stops being used while material at either end still works.

That second property is what makes the fix specific. When both **cost and mid-collection recall** are failing, they are the **same problem**: **volume**. Any option that keeps the full corpus loaded addresses neither, however clever it is about ordering, prompting or emphasis. Only reducing what enters the window per request touches both.

Which Models Have Which Window, and Which Know How Full It Is

Property	Which models
1M-token context window, up to 128k output tokens per request	Claude Opus 5, Opus 4.8, Opus 4.7, Opus 4.6, Sonnet 5, Sonnet 4.6
200k-token window	Other models, including Claude Sonnet 4.5
Context awareness: total token budget injected into the system prompt of every request, plus a remaining-capacity update after each tool call	Claude Sonnet 5, Sonnet 4.6, Sonnet 4.5, Haiku 4.5
No injected capacity tags	Claude Opus 4.7 and later Opus models, Claude Fable 5, Claude Mythos 5

That third row is the one worth holding, because it is counter-intuitive twice over.[1]

Context awareness is not a function of how capable a model is. Some Opus models do not receive the tags while Haiku 4.5 does. A design that assumes the strongest model must also be the best informed about its own remaining capacity has it backwards.

A model without those tags cannot pace itself against the window. If your design depends on the model budgeting its own remaining room, that dependency is model-specific and needs checking rather than assuming.

One pricing fact removes a common worry: on Claude 4.6 and later, the full 1M-token window is available at standard pricing, so a 900k-token request is billed at the same per-token rate as a 9k-token request.[2] There is no premium tier for using the large window. The cost of a big request is the tokens, and the quality cost is the cliff, not a rate change.

Two Ways to Exceed the Window, and They Fail Differently

This distinction is precise and worth memorising exactly, because the two failures need different handling.

Input alone exceeds the window. Every model returns a 400 invalid request error. The request never runs. You find out immediately.[1]

Input plus max_tokens exceeds the window. On Claude 4.5 models and newer the request is *accepted*, runs, and stops with a model_context_window_exceeded stop reason if generation actually reaches the limit.[1]

The second one is the dangerous shape, because it is not an error at submission time. You get a partial result and a stop reason, and a caller that only **checks for HTTP errors** will treat a **truncated answer as a complete one**. Any design generating long output near a full window has to read stop reasons, not just status codes.

Choosing a Tier: Measure, Then Keep Measuring

Model selection is where opinion is cheapest and most expensive.

The documented starting advice is to begin with Claude Opus 5 for complex agentic coding and enterprise work when you are unsure.[3] Published per-million-token pricing sets the shape of the trade-off:

Model	Input	Output
Claude Fable 5	$10	$50
Claude Opus 5	$5	$25
Claude Sonnet 5	$2	$10
Claude Haiku 4.5	$1	$5

Two starting approaches are documented and both are legitimate.[4] **Cost-first** begins with a faster, cheaper model such as Claude Haiku 4.5, which allows quick iteration at lower development cost and is often sufficient. **Capability-first** implements against the strongest starting point for the task and optimises down to more efficient models later.

Now the part the exam actually tests. A tier you can defend is **pinned to an accuracy bar somebody measured**, and it **stays pinned because the measuring continues**. Not settled by assumption, not by splitting the difference between two humans in a room, not inherited from whatever the prototype was built on.

And the counterpart, which decides several items on its own: match capability to the **difficulty of the decision**, not to the **importance of the system**. A high-volume labelling task cannot convert deeper reasoning into accuracy, because there is no reasoning to deepen, and the charge for that depth lands on every message, of which there are millions. Reasoning depth bought for work with no depth in it is a pure cost line.

One structural note about running the comparison. A trial needs precisely **two things** before it can start: a **quality bar everyone has agreed on**, and **one named model to test against** it. Everything else that gets proposed as a prerequisite, signing off the deployment, reading the results, settling the rollout plan, either happens afterwards or belongs to some other decision entirely.

What Thinking and Effort Actually Cost

Thinking tokens are **billed as output tokens** even when the **thinking text is not returned**, and they **count toward max_tokens** alongside the response.[5] That is the sentence that surprises people. Reasoning is not free and not separate; it competes for the same output allowance the answer needs.

On adaptive models, no thinking budget is set, and two controls bound the cost.[6]

max_tokens is a hard ceiling on the request's total output. Generation stops there.

effort is soft guidance about how much of that allowance is spent thinking rather than answering. Its levels run max, xhigh, high, medium, low, and **high is the default**, equivalent to **not setting the parameter** at all.[7] Behaviour runs from always thinking with no constraint on depth at the top, through moderate thinking that may skip simple queries at medium, down to minimised thinking that skips simple tasks where speed matters most at low.[6]

Then the trap that catches agentic designs: in a tool-use loop, **each request has its own max_tokens**, so it does **not bound the whole turn's spend**.[6] Ten round trips means ten separate max_tokens ceilings. If a scenario needs the total bounded, max_tokens is the wrong instrument. **Task budgets** declare a **token allowance for the full agentic loop**, covering **thinking, tool calls, tool results and output**, with a running countdown the model paces against, much as a human does with a deadline.[8] Effort controls thoroughness per step; a task budget caps the total across the loop. They complement each other and neither substitutes for the other.

One migration fact worth knowing: manual extended thinking, which sets a fixed per-request budget with budget_tokens, is documented as deprecated in favour of adaptive thinking controlled by effort. On models that support only extended thinking, including Claude Sonnet 4.5, Opus 4.5 and Haiku 4.5, **requesting adaptive thinking returns a 400 error**.[9] The control you use is a function of the model, not a preference.

Compaction and Context Editing Are Different Instruments

Both manage a growing conversation. They work differently and answer different scenarios.

Compaction automatically summarises older context when a conversation approaches the window limit, which extends effective context length, and it also keeps the active context small because response quality degrades as a conversation grows.[10] The mechanics matter:

- On later requests the API **drops all content blocks prior to the compaction block**. What was not carried into the summary is gone.
- The trigger defaults to **150,000 input tokens** and must be at least **50,000**.
- The default summarisation prompt varies by model and instructs Claude to record the state, next steps and learnings needed to continue in a future window.
- **Custom instructions** supplied through the **instructions parameter replace the default prompt completely** rather than supplementing it.

That last point is a live defect generator. A team writing custom instructions to capture one extra thing usually intends to add to the default behaviour, and instead silently discards the instruction to record state and next steps. If you override, you own the whole prompt.

Context editing selectively clears content from conversation history as it grows, described as active curation on the grounds that context is a finite resource with diminishing returns and that irrelevant content degrades focus.[11] Its cost is specific: **clearing tool results invalidates cached prompt prefixes**. That is why a clear_at_least parameter exists, so a worthwhile minimum is cleared each time and you are not paying a cache invalidation to reclaim a handful of tokens.

The distinction to carry: compaction **summarises** and is triggered by approaching the limit; context editing **removes** and is a curation decision you make deliberately, at a cost to your cache.

Progressive Discovery Against Loading Everything Up Front

The last decision in this chapter is whether material enters the window before the request is understood, or after.

The pattern that wins when it wins is the one that **makes the loaded set a function of the request itself**. Every alternative either fixes the set before anything is known about the request, or pays for all of it regardless.

Deferral is not free, so it needs the conditions spelled out. **Progressive discovery** pays when the **catalogue is mostly irrelevant to any given request**, and when the **token load is already degrading output quality**. Those are the two conditions that make the round trips worth it.

It buys nothing in three cases, and each is a trap in its own right. Material that is **small** is not worth a round trip. Material that is **unchanging** can be cached and left in place. Material that is **required every time** will be fetched every time, so deferring it adds latency and removes nothing.

There is a subtlety worth stating for tool catalogues specifically. The aim is to shrink how much text the model wades through without making the schemas unusable. **Trim the descriptions** so hard that the **tools stop being distinguishable** and you have **swapped a context problem for a selection problem**, and the second one is much harder to spot.

A Filter in Front of a Grown Catalogue Leaves the Catalogue Grown

A tool catalogue that grew one addition at a time eventually starts producing wrong selections, and the first proposal in the room is almost always a **router**: a step ahead of the model that narrows the list before the model ever sees it.

Be precise about what that buys. A router **reduces how many tools are visible per request**. It **changes nothing about the descriptions** the model is choosing between, so **two tools that overlap still overlap** on every request where both survive the filter. The selection errors caused by ambiguity are untouched, and a new

component now has to be right about intent before any reasoning has happened, which moves the failure earlier rather than removing it.

Decomposition is the alternative: **split the catalogue along a seam in the domain** into smaller sets, each one owned and each one coherent read on its own.

Two findings justify that split, and **both have to be present**.

Descriptions that overlap, because overlap is the **mechanism actually producing the wrong selections**, rather than a symptom of the catalogue's size.

A seam that already exists in the domain, so the split produces sets a person can describe in a sentence instead of an arbitrary partition that has to be memorised.

Sheer count is not a finding. Neither is latency, nor spend, nor anybody's discomfort at the length of the list. And an argument that the **router is a poor idea** is not an argument that **decomposition is a good one**: decomposition earns its place from the overlap and the seam, and without them you have reorganised something that was never the problem.

How Wrong Answers Are Built on Model and Context Questions

- **Capacity confused with quality**. The option addresses degraded recall by reordering, re-emphasising or instructing, while the same volume stays loaded. The test: does this **reduce what enters the window** on each request? If not, it cannot fix a **volume symptom**.

- **Depth bought where there is none to buy**. The option raises the model tier or the reasoning depth for a task that is classification, extraction or lookup. The test: is there **multi-step reasoning** here that a stronger model could perform better?

- A **per-request control offered against a whole-loop problem**. The option bounds spend with max_tokens across an agentic loop. The test: how **many requests does this ceiling apply** to?

- **Preference standing in for measurement**. The option settles a tier by seniority, compromise, precedent or what the prototype used. The test: is there a **stated accuracy bar** and a **run against** it?
- **Deferral applied to material that is small, static or always needed**. The test: what **fraction of requests actually need** this, and is it **already hurting quality**?

When the scenario says	The answer is usually	The trap is usually
Recall fails in the middle of a long loaded collection	Reduce what enters the window per request	Reordering, emphasising, or instructing the model to read carefully
Both cost and mid-collection accuracy are degrading	One cause, volume, and one fix	Treating them as two problems needing two changes
A large preamble is cached	It still occupies the window	Concluding that caching freed context room
A model must pace itself against remaining capacity	Check whether that model receives capacity tags at all	Assuming the strongest model is the best informed
Long output is generated near a full window	Read stop reasons, expect model_context_window_ exceeded	Checking only for HTTP errors and treating truncation as completion
Input by itself is too large	A 400 error, before anything runs	Expecting a partial result

When the scenario says	The answer is usually	The trap is usually
A model tier must be justified	A measured accuracy threshold, still being measured	Assumption, compromise, seniority, or whatever the prototype used
High-volume classification or labelling	The cheapest tier that clears the bar	Deep reasoning, which the task cannot convert to accuracy but pays for per message
Total spend across a multi-step loop must be bounded	A task budget covering the full loop	max_tokens, which applies per request
Custom compaction instructions are needed	Write the whole prompt, including state and next steps	Adding one instruction and losing the default behaviour
Tool results are cleared to reclaim room	Expect cache invalidation; clear a worthwhile amount at once	Frequent small clears that pay the cache cost repeatedly
A tool catalogue is mostly irrelevant per request and quality is suffering	Load tools as a function of the request	Fixing the set in advance, or trimming descriptions until tools are indistinguishable
Material is small, unchanging, or needed on every request	Load it up front	Deferring it, which adds round trips and removes nothing

A grown tool catalogue is producing wrong selections | Decompose along a domain seam, on evidence that descriptions overlap | A router in front, which narrows visibility and leaves the overlap exactly where it was |

The habit worth carrying: **before adding anything to a request, ask what it displaces**. The window is working memory, and everything in it competes with everything else.

What to Remember

- **Everything in the request counts** toward the window: system prompt, every message and tool result, images, documents, **tool definitions**, and the turn's output. **Cached prefixes still occupy the window**, because caching changes what you pay for those tokens, not whether they count.

- The context window is **working memory, not storage**, and **accuracy and recall degrade** as the token count grows. The failure is **silent and uneven**, showing up as material in the **middle of a long collection** going unused while both ends still work.

- When cost and mid-collection recall are both failing, they have **one cause: volume**. Only **reducing what enters the window** per request fixes either.

- A **1M-token window** with up to **128k output tokens** is available on Opus 5, 4.8, 4.7, 4.6 and Sonnet 5, 4.6; other models including Sonnet 4.5 have **200k**. On Claude 4.6 and later the full window is **standard pricing**, so a 900k request costs the same per token as a 9k one.

- **Context awareness is model-specific** and **not tied to capability**. Sonnet 5, 4.6, 4.5 and Haiku 4.5 receive their total budget in every system prompt plus a remaining-capacity update after each tool call; **Opus 4.7 and later Opus**, **Fable 5 and Mythos 5** do not.

- **Input alone over the window returns a 400** on every model. **Input plus max_tokens over the window is accepted** on Claude 4.5 and newer and stops with **model_context_window_exceeded** if generation reaches the limit, so a caller checking only HTTP status will **read truncation as completion**.

- A model tier you can defend is **pinned to an accuracy bar somebody measured**, and the **measuring does not stop**. Match capability to how **hard the decision** is, not to how **important the system** is: labelling work cannot turn reasoning depth into accuracy and gets charged for it on every message. A trial needs **two things only**, an **agreed quality bar** and **one named model**.

- **Thinking tokens bill as output** even when not returned and **count against max_tokens**. **max_tokens is a hard cap per request**; **effort is soft guidance** on how much goes to thinking, with levels max, xhigh, high, medium, low and **high as the default**.

- In a tool-use loop **each request has its own max_tokens**, so it does **not bound the turn**. A **task budget covers the whole agentic loop**, thinking and tool calls and results and output, with a countdown the model paces against.

- **budget_tokens manual extended thinking is deprecated** in favour of effort, and requesting **adaptive thinking on an extended-thinking-only model** (Sonnet 4.5, Opus 4.5, Haiku 4.5) **returns a 400**.

- **Compaction** summarises older context near the limit and **drops all content blocks before the compaction block**, triggering at a default of **150,000 input tokens, minimum 50,000**. Custom instructions **replace the default prompt entirely**, so an override that means to add one thing silently loses the instruction to record state and next steps.

- **Context editing removes rather than summarises**, and **clearing tool results invalidates cached prefixes**, which is why **clear_at_least exists**: clear a worthwhile amount at once rather than paying the cache cost repeatedly.

- **Progressive discovery pays** when the catalogue is **mostly irrelevant per request** and the load is **already degrading quality**. It buys nothing for material that is **small, unchanging, or needed every time**. Trimming tool descriptions until the tools stop being distinguishable **swaps a context problem for a selection problem**.

- **Decomposing a grown tool catalogue** requires **two findings, both present**: **descriptions that overlap**, which is the **mechanism producing the wrong selections**, and a **seam already in the domain** to cut along. A **router narrows what is visible per request** and **leaves overlapping descriptions overlapping**, so **arguing against the router is not arguing for decomposition**.